COMPILER DESIGN
PROGRAMMING EXERCISE 6

MAANYA BHARDWAJ
PES2UG23CS311

Lexer.l

```
%{
   #define YYSTYPE char*
   #include <unistd.h>
   #include "y.tab.h"
   #include <stdio.h>
   extern void yyerror(const char *); // declare the error handling function
%}

/* Regular definitions */
digit   [0-9]
letter [a-zA-Z]
id      {letter}({letter}|{digit})*
digits {digit}+
opFraction  (\.{digits})?
opExponent        ([Ee][+-]?{digits})?
number      {digits}{opFraction}{opExponent}
%option yylineno

%%
\/\/(.*) ; // ignore comments
[\t\n] ; // ignore whitespaces
"("             {return *yytext;}
")"             {return *yytext;}
"."             {return *yytext;}
","             {return *yytext;}
"*"             {return *yytext;}
```

```
"+"         {return *yytext;}
";"         {return *yytext;}
"-"         {return *yytext;}
"/"         {return *yytext;}
"="         {return *yytext;}
">"         {return *yytext;}
"<"         {return *yytext;}
{number}    {
                yylval = strdup(yytext);  //stores the value of the number to
be used later for symbol table insertion
                return T_NUM;
            }
{id}        {
                        yylval = strdup(yytext); //stores the identifier to
be used later for symbol table insertion
                        return T_ID;
                    }
.           {} // anything else => ignore
%%
```

Parser.y

```
%{
    #include "quad_generation.c"
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>

    #define YYSTYPE char*

    void yyerror(char* s);
        // error handling function
```

```
	int yylex();
		// declare the function performing lexical analysis
	extern int yylineno;
		// track the line number

	FILE* icg_quad_file;
	int temp_no = 1;
%}


%token T_ID T_NUM

/* specify start symbol */
%start START



%%
START : ASSGN {
					printf("Valid syntax\n");
					YYACCEPT;
				// If program fits the grammar, syntax is valid
				}

/* Grammar for assignment */
ASSGN : T_ID '=' E	{	quad_code_gen(strdup($1), $3, "=", "");  }
	;

/* Expression Grammar */
E : E '+' T  {	$$ = new_temp(); quad_code_gen($$, $1, "+", $3);  }
	| E '-' T	{	$$ = new_temp(); quad_code_gen($$, $1, "-", $3);
}
	| T
	;
```

```
T : T '*' F    {        $$ = new_temp(); quad_code_gen($$, $1, "*", $3);   }
        | T '/' F       {        $$ = new_temp(); quad_code_gen($$, $1, "/", $3);
}
        | F
        ;

F : '(' E ')'   {        $$ = $2;     }
        | T_ID                  {        $$ = strdup($1);  }
        | T_NUM    {        $$ = strdup($1);  }
        ;

%%


/* error handling function */
void yyerror(char* s)
{
        printf("Error :%s at %d \n",s,yylineno);
}


/* main function - calls the yyparse() function which will in turn drive yylex()
as well */
int main(int argc, char* argv[])
{
        icg_quad_file = fopen("icg_quad.txt","w");
        yyparse();
        fclose(icg_quad_file);
        return 0;
}
```

Quad_generation.c

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>

#include "quad_generation.h"

extern FILE* icg_quad_file;
extern int temp_no;

/* Generate new temporary variable */
char* new_temp()
{
   char* temp = (char*)malloc(10);
   sprintf(temp, "t%d", temp_no++);
   return temp;
}

/* Generate quadruple */
void quad_code_gen(char* result, char* arg1, char* op, char* arg2)
{
   printf("( %s, %s, %s, %s )\n", op, arg1, arg2, result);

   // also write to file (optional but usually required)
   fprintf(icg_quad_file, "( %s, %s, %s, %s )\n", op, arg1, arg2, result);
}
```

Quad_generation.h

```c
extern FILE* icg_quad_file;          //pointer to the output file
extern int temp_no;                  //variable to keep track of current temporary count

void quad_code_gen(char* a, char* b, char* op, char* c);
char* new_temp();
```

[run.sh](run.sh)

```
#!/bin/bash

lex lexer.l
yacc -d parser.y
gcc -g y.tab.c lex.yy.c -ll

rm lex.yy.c
rm y.tab.c
rm y.tab.h

./a.out < test_input_1.c
./a.out < test_input_2.c
```

Test_input_1.c

```
x = 9/2 + a-b
```

Test_input_2.c

```
b = c / 6.7 + 12.45 - a * 1234.0
```

Output:

```
maanyabhardwaj@Maanyabhardwaj PE6CD % rm -f lex.yy.c parser.tab.c parser.tab.h pe6

/opt/homebrew/opt/flex/bin/flex lexer.l
/opt/homebrew/opt/bison/bin/bison -d parser.y
gcc parser.tab.c lex.yy.c quad_generation.c -o pe6
./pe6 < test_input_2.c
( /, c, 6.7, t1 )
( +, t1, 12.45, t2 )
( *, a, 1234.0, t3 )
( -, t2, t3, t4 )
( =, t4, , b )
Valid syntax
maanyabhardwaj@Maanyabhardwaj PE6CD % rm -f lex.yy.c parser.tab.c parser.tab.h pe6

/opt/homebrew/opt/flex/bin/flex lexer.l
/opt/homebrew/opt/bison/bin/bison -d parser.y
gcc parser.tab.c lex.yy.c quad_generation.c -o pe6
./pe6 < test_input_1.c
( /, 9, 2, t1 )
( +, t1, a, t2 )
( -, t2, b, t3 )
( =, t3, , x )
Valid syntax
maanyabhardwaj@Maanyabhardwaj PE6CD %
```